FEMU 코드 분석 가이드

발표 준비용 · 기준: `hany-Park/FEMU` master `601ddfe21` (2026-10-07) · BlackBox SSD 모드 (`femu_mode=1`) · 경로는 모두 `~/FEMU/hw/femu/` 기준

선임이 요구한 다섯 질문(I/O 전체 흐름, latency, bandwidth, WAF, GC·Deadline 재설계)에 답하려면 어떤 코드를 봐야 하는지 정리한 것임. 예전 가이드는 FTL이 `bbssd/ftl.c` 한 파일이던 시절 기준이었고, 이 판은 FTL이 여러 파일로 나뉜 최신 master에 맞춰 위치와 설명을 고친 것임.

이동 함수는 같고 파일만 바뀜 수정 예전 설명이 지금 코드와 다름 신규 예전 가이드에 없던 항목 그대로 변화 없음

예전 가이드에서 바뀐 핵심

- **FTL이 파일 하나가 아님.** `bbssd/` 안에서 요청 처리(`ftl-datapath.c`), line·GC(`ftl-line-gc.c`), 지연 (`ftl-media.c`), 매핑(`ftl-map*.c`), 파라미터(`ftl-geom.c`)로 나뉨. `ftl.c` 에는 요청 분기와 WAF 계산 정도만 남음.
- **WAF는 이제 FEMU 자체 기능임.** 카운터와 계산 함수가 들어가 있고, 게스트에서 `nvme get-log` (C0h 로그)로 바로 읽음. "원본에 WAF 코드가 없어서 직접 만들었다"는 말은 더 이상 쓰면 안 됨.
- **GC 희생 정책을 고를 수 있음.** `gc_policy` = greedy(기본), random, cost-benefit, fifo, d-choice. hot/cold 분리(`hot_cold_sep`)도 있음. 예전 가이드에서 "재설계 제안"으로 잡았던 cost-benefit과 hot/cold 분리는 이미 구현돼 있음.
- **요청이 FTL로 넘어가는 지점은 `femu.c` 임.** `bb.c` 는 BBSSD 모드 등록과 데이터 복사 경로이고, FTL 호출은 `femu_ftl_thread()` → `bb_ftl_process_req()` 로 이어짐.
- **`timing-model/` 디렉토리는 없음.** 지연 계산의 실체는 `nand/nand-media.c` 이고, 설명 문서는 `docs/concepts/timing-model.md` 임.
- **공식 설계 문서가 생김.** `docs/design/ftl.md` 가 BBSSD FTL을 자료구조·매핑·line·GC 순으로 설명함. 코드 읽기 전에 같이 펴두면 좋음.

A. I/O 요청 전체 흐름 ★★★

질문 1 "I/O 요청이 들어왔을 때 SSD에서의 전체 흐름". 아래 순서가 실제 코드가 지나가는 순서임.

#	단계 (스레드)	코드 위치
1	게스트가 SQ에 명령을 넣음. poller 스레드가 SQ를 돌며 명령을 꺼냄	`nvme_poller()` → `nvme_process_sq_io()` nvme-io.c:572, :140

#	단계 (스레드)	코드 위치
2	opcode에 따라 모드별 처리로 분기. 읽기·쓰기는 데이터를 RAM 백엔드에 즉시 복사함 (데이터는 이 시점에 이미 저장 끝)	`nvme_io_cmd()` → `bb_io_cmd()` → `nvme_rw()` → `backend_rw()` nvme-io.c:1923 · bbssd/bb.c:395 · nvme-io.c:760, :977
3	요청에 도착 시각을 찍고(`stime = expire_time = now`) FTL용 링에 넣음	`femu_ring_enqueue(n->to_ftl[i], …)` nvme-io.c, nvme_process_sq_io() 안
4	FTL 스레드가 링에서 꺼내 모드별 FTL 호출 (FTL 스레드)	`femu_ftl_thread()` → `femu_ftl_process_req()` femu.c:1881, :1842
5	BBSSD FTL이 opcode별로 분기해 읽기·쓰기 처리 후 지연(ns)을 돌려줌. 끝에서 배경 GC 검사	`bb_ftl_process_req()` → `ssd_read()` / `ssd_write()` bbssd/ftl.c:453 · bbssd/ftl-datapath.c:640, :722
6	`expire_time += lat` 후 poller용 링에 넣음	`femu_ring_enqueue(n->to_poller[i], …)` femu.c, femu_ftl_thread() 안
7	poller가 요청을 `expire_time` 순 우선순위 큐에 넣고, 현재 시각이 그 값을 넘으면 CQ에 완료를 올리고 인터럽트 발생	`nvme_process_cq_cpl()` nvme-io.c:414

파일	우선순위	보는 이유
`bbssd/ftl.h` 그대로 `bbssd/ftl-internal.h` 신규	★★★	자료구조 정의는 여전히 `ftl.h` . 파일이 나뉘면서 FTL 내부 함수 선언과 `should_gc()` 같은 inline 함수는 `ftl-internal.h` 로 감. 둘 다 먼저 봐야 함.
`nvme-io.c` 그대로	★★★	명령 진입(1–3단계)과 완료 게시(7단계).
`femu.c` 수정	★★★	예전엔 ★★☆였지만, 지금은 NVMe 계층과 FTL을 잇는 다리가 여기임(4·6단계). FTL 스레드 루프도 여기 있음.

파일	우선순위	보는 이유
`bbssd/bb.c` 수정	★★☆	BBSSD 모드 등록(`nvme_register_bbssd`)과 읽기·쓰기 명령을 `nvme_rw()` 로 넘기는 부분. FTL과 직접 잇는 다리는 아님.
`bbssd/ftl.c` 수정	★★★	이제 FTL 본체가 아니라 요청 분기(`bb_ftl_process_req`), 배경 GC 호출, WAF 계산 함수가 있는 곳.
`bbssd/ftl-datapath.c` 신규	★★★	`ssd_read` , `ssd_write` , 페이지 하나를 프로그램하는 `ssd_program_lpn` (474), DRAM 쓰기 버퍼. 예전 `ftl.c` 의 요청 처리부가 여기로 옴.
`bbssd/ftl-line-gc.c` 신규	★★★	write pointer, line 관리, GC 전부. D 섹션의 무대.
`bbssd/ftl-media.c` 신규 `nand/nand-media.c` 신규	★★★	지연 계산. B 섹션 참고.
`bbssd/ftl-map.c` 신규	★★☆	L2P 테이블(`maptbl`)과 역매핑(`rmap`) 초기화, 기본 page 매핑. dftl·hybrid·fast는 `ftl-map-*.c` .

읽는 순서: `ftl.h` → `nvme-io.c` (1–3) → `femu.c` (4, 6) → `ftl.c` (5) → `ftl-datapath.c` → `ftl-line-gc.c` · `ftl-media.c` → `nvme-io.c` (7). 구조체를 먼저 보고, 요청이 지나가는 순서대로 따라가면 흐름이 끊기지 않음.

B. Latency · Bandwidth 계산 ★★★

질문 2·3. 파일보다 특정 함수와 필드가 중요함.

함수 / 필드	보는 이유
`ssd_advance_status()` 이동 `bbssd/ftl-media.c:187`	FTL이 지연을 물어보는 입구. NAND 명령(읽기·쓰기·소거)을 매체 연산으로 바꾸고 횟수를 센 뒤 `nand_media_op()` 를 부름. 예전엔 계산까지 이 함수가 했지만 지금은 넘겨주기만 함.
`nand_media_op()` 신규 `nand/nand-media.c:385`	실제 지연 계산. 기본 설정(채널 지연 0)에서는 LUN의 가용 시각과 요청 시각 중 늦은 쪽에 연산 지연을 더함. 이 함수를 설명할 수 있으면 latency 질문은 끝남.

함수 / 필드	보는 이유
`nand_lun.next_lun_avail_time` 그대로	LUN이 언제 비는지. 같은 LUN에 몰린 요청이 직렬화되는 근거이고, 병렬성 설명의 핵심.
`ssd_channel.next_ch_avail_time` 수정	채널 가용 시각. 이제 `ch_xfer_lat` > 0이면 채널 버스 단계까지 모델링함(예전엔 무시됨). 기본값은 0.
`ssd_write()`의 `maxlat` bbssd/ftl-datapath.c:722	요청 하나가 여러 페이지면 페이지별 지연 중 최댓값이 요청 지연이 됨. 페이지가 여러 LUN에 흩어지면 병렬로 처리되는 이유.
`struct ssdparams` + `ssd_init_params()` 이동 bbssd/ftl.h:137 · bbssd/ftl-geom.c:225	`pg_rd_lat`, `pg_wr_lat`, `blk_er_lat`, `ch_xfer_lat` 같은 상수. 실행 스크립트 값이 `bb_params`를 거쳐 여기서 `ssdparams`로 들어감.
`docs/concepts/timing-model.md` `docs/design/nand-timing.md` 수정	예전 가이드의 `timing-model/` 디렉토리 대신 볼 문서. 셀 타입별 지연(`nand_cell_type`), 멀티플레인(`mp_program`), PCIe 링크 모델(`pcie_bandwidth_mbps`)처럼 "FEMU가 어디까지 모델링하는가"를 답할 때 씀.

핵심 한 줄 (지금 코드에서도 그대로 맞음)

```
완료 시각 = max(요청 시각, LUN 가용 시각) + 연산 지연(rd / wr / er)
```

bandwidth를 따로 계산하는 변수는 없음. 위 완료 시각들이 쌓인 결과를 호스트(fio)가 측정한 값이 bandwidth임. "FEMU는 대역폭을 직접 정하지 않고, 지연 모델의 결과로 대역폭이 정해진다"가 답. 단, PCIe 링크 모델을 켜면 링크 대역폭 제한이 완료 단계(7단계)에 추가로 걸림.

C. 핵심 자료구조 ★★★

모두 `bbssd/ftl.h`. 발표 중 질문에 바로 답하려면 외울 정도로 봐야 함.

구조체	역할 / 보는 이유
`struct ssd` 수정 ftl.h:435	SSD 전체를 대표하는 최상위 구조체. 예전보다 커졌음: WAF 카운터(`host_write_pages`, `nand_write_pages`, `gc_write_pages`, `total_erases`), 쓰기 버퍼, 읽기 캐시, 매핑·GC 정책 포인터가 추가됨.
`struct ssdparams` 그대로	용량·레이아웃·지연·GC 임계값 등 모든 설정값.

구조체	역할 / 보는 이유
`struct ppa` 그대로 `ftl.h:69`	물리 주소(ch/lun/pl/blk/pg/sec)를 비트필드로 나눈 것. 주소 변환 설명의 핵심. 필드 폭은 넓어졌음.
`nand_page` / `nand_block` / `nand_plane` / `nand_lun` / `ssd_channel` 그대로	NAND 계층을 그대로 코드로 옮긴 것. 아키텍처 그림과 1:1로 맞춰 보여줄 수 있음. `nand_block`에 `erase_cnt`, `read_cnt`가 있음.
`struct line` / `struct line_mgmt` 수정 `ftl.h:223, :251`	GC 단위는 block이 아니라 line(모든 채널·LUN·plane에서 같은 번호의 블록 묶음, 사실상 superblock). GC 한 번에 소거되는 블록 수 = `nchs × luns_per_ch × pls_per_lun` (기본 스크립트는 8×8×1 = 64). `close_time`, `close_seq`는 cost-benefit·FIFO 정책용으로 추가됨.
`struct write_pointer` 수정	다음에 쓸 위치. out-of-place update가 구현되는 곳. 이제 기본 `wp` 외에 스트림별, hot/cold(`hot_wp`), 로그 블록(`log_wp`)용 포인터가 따로 있음.
`maptbl` / `rmap` 그대로	L2P(논리→물리)와 P2L(물리→논리) 테이블. GC가 옮길 페이지의 논리 주소를 찾으려면 `rmap`이 필요함.

`femu_ftl_policy_ops`
`femu_mapping_ops` 신규
`ftl.h:355, :381`
	GC 희생 정책과 매핑 방식을 갈아끼우는 함수 테이블. "정책을 바꾸려면 어디를 고치나"에 대한 답이 여기 있음. 새 정책 하나를 추가하는 것이 재설계 실험의 가장 깔끔한 방법임.

D. GC 재설계 (write-intensive 지연 개선) ★★★

질문 4. GC 코드는 전부 `bbssd/ftl-line-gc.c` 로 옮겨졌음.

함수	보는 이유
`should_gc()` / `should_gc_high()` 이동 `bbssd/ftl-internal.h:205`	GC 발동 조건. `free_line_cnt <= gc_thres_lines` 한 줄. high는 강제 GC 기준(`gc_thres_pcent_high`).
배경 GC 호출 수정 `bbssd/ftl.c`, `bb_ftl_process_req()` 끝 (≈578)	요청을 하나 처리할 때마다 `should_gc()`이면 `do_gc(ssd, false)`. 장치가 쉴 때 도는 방식이 아니라 매 요청 뒤에 도는 방식임. 예전 가이드는 `ftl_thread()`가 결정한다고 했지만 지금은 여기임.

함수	보는 이유
강제(전경) GC 호출 신규 `bbssd/ftl-datapath.c:743 (ssd_write 시작), :549`	쓰기 요청 처리 전에 `should_gc_high()` 인 동안 `do_gc(ssd, true)` 를 반복. 쓰기 지연이 튀는 직접 원인이므로 재설계 제안의 근거 위치.
`do_gc()` 이동 `bbssd/ftl-line-gc.c:946`	희생 line 선택(`ssd->policy->select_victim_line`) → `reclaim_line()` . 이제 짧아졌고 실제 절차는 `reclaim_line()` 에 있음.
`reclaim_line()` 신규 `ftl-line-gc.c:902`	GC 본체. 모든 ch/lun/pl에 대해 유효 페이지 복사 → 블록 소거 → line을 free로 반환.
`select_victim_line*()` 수정 `ftl-line-gc.c:641–759`	greedy(641), random(663), cost-benefit(686), fifo(737), d-choice(759). `gc_policy` 옵션으로 고름. 예전 가이드는 "Greedy만 있다"였음.
`clean_one_block()` / `gc_read_page()` / `gc_write_page()` 이동 `ftl-line-gc.c:838, :523, :595`	유효 페이지를 실제로 옮기는 부분. WAF가 생기는 지점.
`ssd_erase_lun_block()` / `mark_block_free()` / `mark_line_free()` 이동 `ftl-line-gc.c:468, :499, :860`	소거 지연 반영과 소거 횟수 집계(`erase_cnt` , `total_erases`), line을 free 풀로 반환.

WAF 계산 (질문 5) 수정

FEMU 자체에 들어가 있음. 계산 함수는 `ssd_waf_x1000()` (`bbssd/ftl.c:176`)이고, 카운터는 다음 위치에서 증가함.

- `host_write_pages` : 호스트가 쓴 페이지 (`ftl-datapath.c:754` , ssd_write)
- `nand_write_pages` : 실제 NAND에 프로그램된 사용자 페이지 (`ftl-datapath.c:507`)
- `gc_write_pages` : GC가 옮긴 페이지 (`ftl-line-gc.c:557`)

```
WAF = (nand_write_pages + gc_write_pages) / host_write_pages
```

쓰기 버퍼가 꺼져 있으면(기본) `nand_write_pages = host_write_pages` 라서 예전 식과 같음. 게스트에서 `nvme get-log` C0h 로그로 읽고, 한 번의 실험 WAF는 실험 전후 값의 차이로 계산함 (`docs/guides/measuring.md`).

재설계 제안을 다시 잡아야 함. cost-benefit과 hot/cold 분리는 이미 있으므로 "새로 넣자"가 아니라 "기존 정책들을 같은 워크로드로 비교해 보니 이렇다"가 출발점이 됨. 아직 없는 것은 **장치가 쉴 때 미리 도는 idle-time GC**와 **강제 GC를 쪼개서 쓰기 지연을 분산하는 방식**이고, 제안의 무대는 위 표의 배경·강제 GC 호출 위치임.

E. Deadline 임박 요청 처리 (Request Queue 재설계) ★★☆

질문 5. 아직 손대지 않은 영역이라 가장 약한 부분. 발표 전에 시간을 가장 많이 들여야 함.

위치	보는 이유
`nvme_process_sq_io()` `nvme-io.c:140`	SQ에서 꺼낸 순서대로 `to_ftl` 링에 넣음. 이 단계에 우선순위 개념은 없음.
`femu_ring` / `to_ftl` / `to_poller` `inc/rte_ring.h` · `lib/rte_ring.c`	NVMe 계층과 FTL 스레드 사이의 FIFO 링 버퍼. deadline 기반 재정렬을 넣으려면 이 구조를 바꿔야 함.
`femu_ftl_thread()` 루프 이동 `femu.c:1881`	링에서 요청을 꺼내는 지점. EDF(Earliest Deadline First) 같은 스케줄링을 끼워넣을 자리.
`nvme_process_cq_cpl()` 신규 `nvme-io.c:414`	완료 쪽은 이미 `expire_time` 순 우선순위 큐(`lib/pqueue.c`)를 씀. 다만 이것은 "언제 완료를 알릴지"만 정할 뿐, FTL이 요청을 처리하는 순서는 바꾸지 않음. "FEMU는 완전 FIFO다"라고 말하면 틀리므로 이 구분을 정확히 해야 함.
`NvmeRequest` `nvme.h:1610`	요청 하나가 들고 다니는 정보(`stime`, `expire_time`, `reqlat` 등). deadline 필드는 여전히 없음 → "필드 추가 + 정렬 삽입"이라는 구체적 제안이 나옴.

답변 방향 (3단 구성)

① FTL이 요청을 처리하는 순서는 FIFO이고, 완료 통보만 `expire_time` 순으로 정렬됨 → ② `NvmeRequest`에 deadline 필드를 추가하고, `to_ftl` 링 대신 이미 코드에 있는 `pqueue`를 deadline 기준으로 써서 FTL 처리 순서를 바꿈 → ③ 단, 강제 GC는 `ssd_write()` 안에서 요청을 막고 돌기 때문에, 재정렬만으로는 부족하고 D(GC 재설계)와 같이 가야 함.

F. 실행 · 설정 ★☆☆

파일	보는 이유
`scripts/run-blackbox.sh` 수정	원본은 `hw/femu/scripts/`에 있고 `femu-copy-scripts.sh`로 `build-femu/`에 복사해 씀. master 기본값은 `blks_per_pl=256`, `ssd_size=12288`, `gc_thres_pcent=75`. 지금 `build-femu/`에 있는 건 예전 사본이므로 다시 복사해야 함.
파라미터 → 코드 연결 신규	스크립트 값 → `femu.c`의 `DEFINE_PROP_*` (예: `gc_thres_pcent`는 femu.c:2996) → `n->bb_params` → `ssd_init_params()` (ftl-geom.c:225) → `ssdparams`. 발표에서 설정→코드 흐름을 보여줄 때 씀.
`docs/properties.md` 신규	모든 장치 옵션 목록. `gc_policy`, `hot_cold_sep`, `buffer_size`, `mapping` 같은 새 옵션을 여기서 확인함.

G. 참고용 (시간 남으면) ★☆☆

위치	보는 이유
`docs/design/ftl.md` 신규	BBSSD FTL 공식 설계 문서. 시간이 부족하면 이것부터 읽는 게 가장 효율적임 (사실상 ★★★).
`backend/dram.c` 그대로	실제 데이터를 저장하는 RAM 백엔드. "데이터는 RAM에, 지연만 모델링한다"의 근거.
`bbssd/ftl-cache.c`, `ftl-map-cmt.c`, `ftl-map-hybrid.c`, `ftl-map-fast.c` 신규	읽기 캐시와 DFTL·BAST·FAST 매핑. 기본값에서는 꺼져 있음.
`bbssd/ftl-fdp.c` 신규	FDP(Flexible Data Placement) 모드 전용 FTL·GC. 기본 경로와 별개라 섞지 말 것.
`zns/`, `ocssd/`, `kvssd/`, `cxlssd/` 수정	다른 모드. 모드마다 FTL이 따로 있음. KV-SSD, CXL-SSD 모드가 새로 생김.

추천 읽기 순서 (총 3일 정도)

#	할 일	분량	목표
1	`docs/design/ftl.md` 훑기 + `ftl.h`, `ftl-internal.h` 통독	반나절	구조체 지도와 파일 지도를 같이 머리에 넣기.
2	I/O 경로 추적: `nvme-io.c` → `femu.c` → `ftl.c` → `ftl-datapath.c`	하루	A 섹션 7단계를 손으로 그릴 수 있게 되기.
3	`ssd_advance_status()` → `nand_media_op()` 정독	반나절	latency·bandwidth 질문 대비.
4	`do_gc()` → `reclaim_line()` + 배경·강제 GC 호출 위치 + 희생 정책 5개	반나절	GC 재설계 답변과 WAF 설명 완성.
5	큐 구조: `to_ftl` 링, `pqueue`, `NvmeRequest`	반나절	deadline 재설계 답변. 가장 약한 부분이니 시간 확보.

빠른 시작 명령

모두 `~/FEMU/hw/femu`에서 실행. 현재 master에서 결과가 나오는 것을 확인함.

```
# 0) 작업 위치
cd ~/FEMU/hw/femu

# 1) BBSSD FTL 파일 구성 보기
ls bbssd/

# 2) 구조체 목록 (지도 먼저 그리기)
grep -n "^struct\|^typedef struct" bbssd/ftl.h

# 3) 요청 처리·GC 파일의 함수 목록
grep -n "^[a-z].*(" bbssd/ftl-datapath.c bbssd/ftl-line-gc.c | grep -v ";"

# 4) I/O 경로의 스레드 진입점
grep -n "^void \*nvme_poller\|^static void nvme_process_sq_io\|^static void nvme_pro
cess_cq_cpl" nvme-io.c
grep -n "^static void \*femu_ftl_thread\|^static uint64_t femu_ftl_process_req" fem
u.c

# 5) 지연 계산 함수 본문 (B 섹션)
sed -n '/^uint64_t ssd_advance_status(/,/^}/p' bbssd/ftl-media.c
sed -n '/^NandOpCompletion nand_media_op(/,/^}/p' nand/nand-media.c

# 6) GC 관련 함수 (D 섹션)
grep -n "should_gc\|do_gc\|select_victim_line\|reclaim_line\|clean_one_block\|gc_wri
te_page\|mark_line_free" \
    bbssd/ftl-internal.h bbssd/ftl-line-gc.c

# 7) WAF 카운터가 증가하는 위치
grep -rn "host_write_pages +=\|nand_write_pages++\|gc_write_pages++\|total_erases++"
bbssd/

# 8) (게스트 안에서) WAF x 1000 읽기
sudo nvme get-log /dev/nvme0 --log-id=0xc0 --log-len=512 -b | od -An -t u4 -N 4
```

핵심 조언 3가지

① `ftl.h`와 `docs/design/ftl.md`를 먼저 볼 것

구조체 지도 없이 코드를 읽으면 계속 막힘. 파일이 여러 개로 나뉜 지금은 공식 문서로 파일 지도까지 먼저 잡는 게 빠름.

② 기본 경로만 볼 것

`femu_mode=1`, FDP 꺼짐, 쓰기 버퍼 꺼짐, `mapping=page`, `gc_policy=greedy`가 기본값. 코드 곳곳에 FDP·스트림·버퍼 분기가 있는데, 처음엔 기본값에서 타지 않는 분기는 건너뛰어야 흐름이 보임.

③ "BBSSD FTL"도 여러 파일임

모드마다 FTL이 따로 있을 뿐 아니라, BBSSD FTL 자체도 7개 이상의 파일로 나뉨. 발표에서는 "최신 master의 BBSSD 기본 경로를 봤다"고 범위를 밝히는 게 정확함.
